

Indice

1	Dischi e Partizioni	2
2	Virtualizzazione	3
2.1	Docker	3
3	Linguaggio C	5
3.1	Permessi	6
4	Processi	8
4.1	Lista D'Ambiente	8
4.2	Esecuzione di un programma	8
4.3	Identificazione dei Processi	9
4.4	Creazione dei Processi	9
4.5	Race Condition	9
4.6	Esecuzione di un programma	9
4.7	Passare l'ambiente ad exec	9
4.8	Terminazione di un processo	10
5	Segnali	11
5.1	Inviare segnali	11
6	PIPE	12
6.1	Creazione di una Pipe	12
6.2	popen e pclose	13
6.3	Pipe con nome	13
7	Thread	14
7.1	Utilizzo	14
7.2	Terminazione dei thread	14
7.3	Condivisione della memoria	14
8	Concorrenza	15
8.1	Tipi di sincronizzazione	15
8.2	Mutex Posix	15
8.3	Deadlock	15
8.4	Condition Variable	16
9	Socket	17
9.1	Domini e Stili di comunicazione	17
9.2	Creazione	17
9.3	Invio dati a un socket	17
9.4	Ricezione dati da un socket	17

1 Dischi e Partizioni

Disco rigido diviso in unità più piccole chiamate [partizioni](#). Possiamo avere:

- Partizioni [primarie](#) = contengono generalmente il sistema operativo e i file di avvio.
- Partizioni [secondarie](#) = contengono generalmente dati.

Nello schema di partizionamento originale si avevano solo 4 partizioni primarie.

Si possono dividere in altre partizioni più piccole chiamate [partizioni logiche](#).

La partizione primaria che viene divisa prende il nome di [partizione estesa](#).

[Windows](#) denomina le partizioni con le lettere e le chiama unità.

[Linux](#) denomina le partizioni usando 3 lettere + 1 numero.

- La prima lettera può essere **h** per IDE o **s** per SCSI/SATA.
- La terza lettera può essere **a-d** per IDE o un numero illimitato per SCSI/SATA.
- Il numero indica il numero di partizioni.
 - I numeri da 1 a 4 indicano le partizioni primarie.
 - Le partizioni logiche si contano sempre da 5 in poi.

La [Tabella delle Partizioni](#) si trova nel MBR (*Master Boot Record*) del disco rigido. Contiene:

1. Flag della partizione attiva.
2. Indirizzo della partizione in formato CHS.
3. Tipo di partizione
4. Indirizzo in formato CHS dell'ultimo settore.
5. Numero di settori del disco che prededono la partizione.
6. Numero di settori del disco che compongono la partizione.

2 Virtualizzazione

Si basa sulla creazione della versione virtuale di una risorsa fornita fisicamente.

Possiamo avere quindi su una sola macchina (host) diversi sistemi operativi (sistemi guest).

Tipi di virtualizzazione sono:

1. **Hardware** = un esempio è la VM (computer virtuale che si comporta esattamente come un computer fisico). Ha come elemento l'**Hypervisor**, ovvero un software che gestisce le risorse hardware (RAM, CPU,...).
2. **Completa** = Hypervisor simula un hardware completo per ogni VM. Il SO ospite non sa di essere virtualizzato. Un esempio è VirtualBox.
3. **Paravirtualizzazione** = Hypervisor offre delle API. Il SO sa di essere virtualizzato e comunica con l'Hypervisor tramite le API.

2.1 Docker

Piattaforma open-source che permette di creare, distribuire e gestire applicazioni in ambienti isolati chiamati **container** (unità software che contengono tutto il necessario per eseguire l'applicazione, come codice, librerie, dipendenze,...).

I vantaggi di Docker sono:

- Flessibilità
- Efficienza
- Portabilità
- Velocità
- Sicurezza

Il **Dockerfile** è un file di testo che contiene le istruzioni per creare un'**immagine Docker**. La sua sintassi è composta dai comandi:

- FROM
- RUN
- WORKDIR
- COPY
- USER
- CMD

Il **Docker Registry** è dove vengono immagazzinate immagini Docker. Può essere privato/pubblico o locale/cloud.

Il **Docker Hub** è esattamente come GitHub.

Il **Docker Volume** è una cartella condivisa tra il container e il computer host per immagazzinare i dati in maniera persistente.

Il [Docker Compose](#) permette di gestire i propri container salvandone la configurazione di istanziamiento in un unico file in formato YAML.

Quando si aggiunge un container, basta salvare la sua configurazione nel file `docker-compose.yaml`.

Usare il Docker Compose è un processo in tre fasi:

1. Scrivere il Dockerfile
2. Configurare i container in `docker-compose.yaml`
3. Eseguire il comando `docker compose up`.

I vantaggi Docker Compose sono:

- File Compose
- Servizi
- Reti
- Volumi
- Scalabilità

3 Linguaggio C

I file UNIX sono strutture costituite da 3 elementi:

- **Nome**
- **Inodo** = struttura dati che contiene informazioni necessarie al SO per la sua gestione e gli indirizzi per accedere ai dati contenuti nel file.
- **Dati**

Quando un processo si riferisce a un file per nome (path assoluto o relativo), il kernel divide il path in parti (tipo matriosca) e controlla i permessi delle varie directory del percorso.

- Se il processo vuole creare un nuovo file $\implies$ kernel **assegna** un inodo non utilizzato al nuovo file.
- Se il processo vuole accedere a un file $\implies$ kernel **ritrova** l'inodo del file, se esiste.

In entrambi i casi, alla file il kernel restituisce il **file descriptor** (un intero non negativo).

Il metodo **open(pathname, flags, mode)**

- **flags** indica la modalità di apertura del file (sola lettura, sola scrittura, ...)
- **mode** è opzionale e serve per la creazione del file.
- Restituisce il file descriptor o -1 in caso di errore.

Il metodo **create(pathname, mode)**

- Se il file esiste $\implies$ troncato a zero.
- Restituisce il file descriptor o -1 in caso di errore.

Il metodo **read(fd, buf, count)**

- Restituisce il numero di byte letti o -1 in caso di errore.
- La lettura inizia dalla posizione corrente del file e aumenta del numero di byte letti.

Il metodo **write(fd, buf, count)**

- Restituisce il numero di byte scritto o -1 in caso di errore.
- La scrittura inizia dalla posizione corrente del file e aumenta del numero di byte scritti.

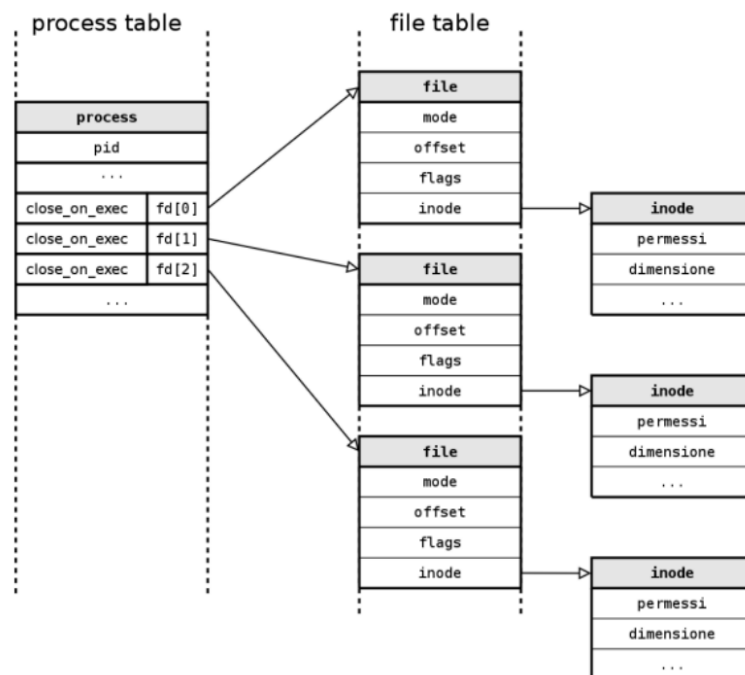
Il metodo **close(fd)**

- Restituisce 0 o -1 in caso di errore.

Il metodo **lseek(fd, offset, whence)** sposta la posizione corrente - offset - di lettura o scrittura all'interno di un file aperto.

- Restituisce la nuova posizione corrente del file o -1 in caso di errore.
- **offset** indica il numero di byte da spostare.
- **whence** indica da dove partire con lo spostamento (se l'offset è relativo all'inizio del file, o alla fine o alla posizione corrente del file).

Ogni processo mantiene una tabella dei propri file aperti ([file descriptor table](#)) che contiene un puntatore ad un entry nella tabella dei file aperti ([file table](#)) del kernel. Ogni elemento della file table contiene un puntatore alla struttura dati [v-node](#) che contiene le informazioni sull'inodo del file.



- Ogni utente ha un identificatore numerico univoco, detto [User ID](#).
- Ogni gruppo ha un identificatore numerico univoco detto [Group ID](#).
- Ad ogni file viene associato un User ID e un Group ID.
- Ogni processo possiede:
 - [Real User ID](#)
 - [Effective User ID](#)
 - [Real Group ID](#)
 - [Effective Group ID](#)

3.1 Permessi

Per avere [accesso ad un file](#) è necessario:

- **Permessi di esecuzione** sulla directory stessa e su quelle padre.
- **Permessi di lettura** sul file stesso e sulle directory padre.
- **Permessi specifici** sul file.

Per [creare un file](#) bisogna avere i **permessi di scrittura** sulla directory.

Per [cancellare un file](#) bisogna avere i **permessi di scrittura** sulla directory stessa.

Il metodo `access(pathname, mode)`:

- Controlla se il file specificato da `pathname` è accessibile con le modalità specificate da `mode`.
- Restituisce 0 o -1 in caso di errore.

Il metodo `chmod pathname, mode`:

- Imposta i permessi di accesso al file specificato da `pathname` in base al parametro `mode`.

Il metodo `fchmod(fd, mode)`:

- Imposta i permessi di accesso al file associato a `fd` in base al parametro `mode`.

Il metodo `chown(pathname, owner, group)`:

- Imposta l'owner ed il group owner del file specificato da `pathname` in base ai parametri `owner` e `group`.

Il metodo `fchown(fd, owner, group)`:

- Imposta l'owner ed il group owner del file associato a `fd` in base ai parametri `owner` e `group`.
- Restituisce 0 in caso di successo, o -1 altrimenti.

4 Processi

Ad ogni programma viene passata una [lista d'ambiente](#).

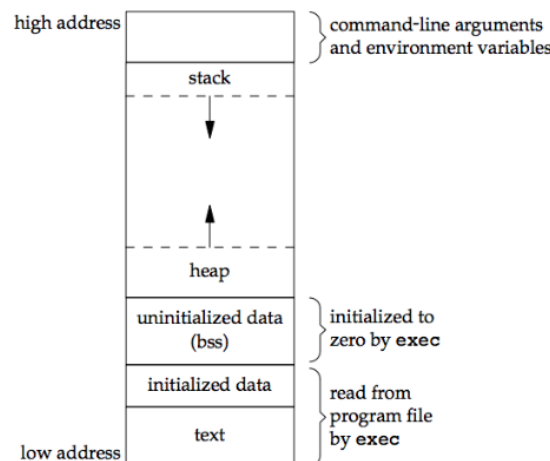
È un array di stringhe nella forma **nome=valore** con l'aggiunta di `NULL` alla fine dell'array.

Per accedere alla lista si possono usare i metodi:

- `getenv` = vuole in input il nome della variabile d'ambiente e restituisce il suo valore.
- `putenv` = vuole in ingresso una stringa **nome=valore** e la aggiunge alla lista d'ambiente.

4.1 Lista D'Ambiente

- [Testo](#) = codice del programma.
- [Dati inizializzati](#) = variabili globali e statiche inizializzate.
- [Dati non inizializzati](#) = variabili globali e statiche non inizializzate.
- [Heap](#) = contiene le variabili allocate dinamicamente.
- [Stack](#) = contiene variabili locali e parametri delle funzioni.



4.2 Esecuzione di un programma

La [fine dell'esecuzione](#) di un programma può avvenire in più modi:

- **Normal Exit** = programma termina normalmente, restituendo un valore alla funzione `main`.
- **Error Exit** = programma termina normalmente, restituendo un valore diverso da 0 alla funzione `main`.
- **Exit** = programma termina normalmente, restituendo un valore alla funzione `exit`.
- **Abort** = programma termina in modo anomalo, restituendo un valore alla funzione `abort`.
- **Signal** = programma termina in modo anomalo a causa di un segnale.

4.3 Identificazione dei Processi

I processi UNIX si dividono in [processi di sistema](#) e [processi utente](#).

Si identificano con un [PID](#) (Process IDentifier), che è un intero positivo.

- Il PID 0 è per il processo padre.
- Il PID 1 è per il processo `init`, il primo ad essere eseguito dal kernel.
- Il PID 2 è per il processo `pager` che gestisce la memoria virtuale.

Ogni Processo utente eredita una serie di identificativi:

- UID (User IDentifier)
- EUID (Effective User IDentifier)
- GID (Group IDentifier)
- EGID (Effective Group IDentifier)
- PPID (Parent Process IDentifier)

4.4 Creazione dei Processi

Avviene tramite una delle due:

- Il metodo `fork` = crea un processo figlio. Restituisce il PID del nuovo processo al processo padre e 0 al processo figlio. Il processo figlio *eredita* lo spazio di indirizzamento del padre.
- Il metodo `vfork` = crea un processo figlio che *condivide* lo spazio di indirizzamento con il padre. È più veloce ma più pericolosa.

4.5 Race Condition

Si verifica quando due processi accedono contemporaneamente alla stessa risorsa senza sincronizzazione.

L'ordine di esecuzione dei processi non è deterministico, quindi non è possibile prevedere il risultato dell'esecuzione.

4.6 Esecuzione di un programma

L'esecuzione di un nuovo programma si ottiene con la chiamata ad una delle funzioni `exec`:

- Sostituisce il codice del processo chiamante con quello del programma passato come argomento.
- Le aree di memoria `text`, `data` e `stack` vengono sostituite con quelle del nuovo programma.

4.7 Passare l'ambiente ad exec

Per accedere all'ambiente di un programma C, è sufficiente aggiungere il parametro `envp` a quelli del `main`.

Oppure è possibile usare la variabile globale `environ`.

4.8 Terminazione di un processo

Quando un processo termina, viene inviato il segnale `SIGCHLD` al padre.

Il processo diventa uno "zombie" fino a quando il padre non chiama una `wait/waitpid`.

I modi per terminare un processo in modo normale sono:

- `int exit(int status)` = termina il processo chiamante, restituendo un valore.
- `return` = termina la funzione `main`, restituendo un valore.

I modi per terminare un processo in modo anomalo sono:

- `void abort(void)` = termina un processo chiamante, inviando il segnale `SIGABRT`.
- Ricevendo determinati segnali.

Cosa succede se il padre termina prima del figlio?

- Il figlio viene ereditato dal processo `init` che vuole evitare che il figlio diventi orfano (senza `PPID`).

Cosa succede se il figlio termina prima del padre?

- Generalmente il padre aspetta mediante la chiamata di sistema `wait/waitpid`.
- Se il padre non aspetta, il figlio diventa uno zombie.

Il metodo `wait(status)` permette di aspettare la terminazione di un processo figlio.

Il metodo `waitpid(pid, status, options)` permette di aspettare la terminazione di un processo figlio specifico.

- `pid` contiene il PID del processo figlio.
- `status` contiene il valore restituito dal processo figlio (se il processo figlio è terminato normalmente o a causa di un segnale, ecc...)

5 Segnali

- Viene inviato ad un processo per notificare un evento.
- Ogni segnale ha un proprio nome che inizia per **SIG**.
- Sono identificati da un numero intero positivo.

Un segnale può essere gestito in diversi modi:

- **Ignorare il segnale:**
 - Il segnale viene ignorato e non viene svolta alcuna azione.
 - I segnali **SIGKILL** e **SIGSTOP** non possono essere ignorati.
- **Catturare il segnale:**
 - Il segnale viene catturato e viene eseguita una funzione di gestione.
 - La funzione di gestione viene registrata con la funzione **signal** o **sigaction**.
 - La funzione di gestione viene eseguita in un nuovo processo, quindi non può accedere alle variabili locali del processo chiamante.

5.1 Inviare segnali

Viene utilizzato il metodo **kill(pid, sig)**:

- **pid** è il PID del processo destinatario.
- **sig** è il numero del segnale da inviare.

6 PIPE

È un [canale di comunicazione](#) tra due processi.

- L'output di un processo diventa l'input del secondo.
- Esempio di questo meccanismo è l'utilizzo dell'operatore `|`.

Le caratteristiche sono:

1. È unidirezionale.
2. Ha una capacità massima.
3. È un file speciale.
4. La pipe (classica) necessita che i processi abbiano un antenato in comune.
5. Si dice "chiusa" quando i processi che la usano hanno terminato.

6.1 Creazione di una Pipe

Si usa il metodo `pipe(fd[2])`

- `fd[0]` è descrittore di lettura.
- `fd[1]` è descrittore di scrittura.
- L'output di `fd[1]` è l'input di `fd[0]`.
- Restituisce 0 in caso di successo, -1 altrimenti.

Una volta creata, la pipe può essere usata come un file.

Il metodo `read` rimuove i dati dalla pipe.

- Se l'estremo di `write` è **aperto**:
 - Ottiene dati disponibili, ritornando il numero di byte letti.
 - Successive chiamate si bloccano fino a quando nuovi dati non saranno disponibili.
- Se l'estremo di `write` è **chiuso**:
 - Ottiene dati disponibili, ritornando il numero di byte letti.
 - Successive chiamate ritornano 0 per indicare la fine del file.

Il metodo `write` aggiunge dati alla pipe.

- Se l'estremo di `read` è **aperto**:
 - Scrive i dati, ritornando il numero di byte scritti.
 - Successive chiamate si bloccano fino a quando non ci sarà spazio disponibile.
- Se l'estremo di `read` è **chiuso**:
 - Ritorna un errore `SIGPIPE`.

6.2 popen e pclose

Semplificano il lavoro manuale di gestione dei processi e delle pipe.

Il metodo `popen`:

1. Crea una pipe
2. Esegue una `fork()`
3. Invoca la shell

Restituisce un puntatore a `FILE` in caso di successo, `NULL` altrimenti.

Il metodo `pclose`:

- Aspetta la terminazione del processo figlio (fa una `waitpid` implicita).

Restituisce il valore di ritorno del comando in caso di successo, -1 altrimenti.

6.3 Pipe con nome

È un file speciale che permette la comunicazione tra processi non imparentati. Devono condividere il nome della FIFO.

Per creare una FIFO si usa il metodo `mkfifo(pathname, mode)`:

- `pathname` è il nome della FIFO.
- `mode` è la modalità di apertura.
- Restituisce 0 in caso di successo, -1 altrimenti.

Una volta creata, può essere usata come file.

7 Thread

Sono **unità esecutive indipendenti** all'interno di un processo, caratterizzate da:

- **thread id**
- **Insieme di registri**
- **pila** per le variabili locali e l'indirizzo di ritorno.
- **Maschera dei segnali**
- **Priorità di esecuzione**

Tutti i thread relativi ad uno stesso processo ne condividono:

- Spazio degli indirizzi
- File aperti
- Segnali pendenti
- Credenziali

7.1 Utilizzo

Quando un **thread viene creato**, viene associato a un pezzo di codice.

- Se il processo ospite è *single-threaded*, il thread è associato all'intero programma.
- Altrimenti, è associato ad una funzione del programma chiamata **funzione di start**

Quando un **thread termina**, il programmatore deve preoccuparsi affinché tutte le risorse impiegate dal thread vengano rilasciate al sistema.

Quando un **programma viene mandato in esecuzione** tramite la chiamata **exec**, viene creato un singolo thread detto **thread principale**. Gli altri devono essere creati esplicitamente.

7.2 Terminazione dei thread

La terminazione di un thread può avvenire in tre diverse circostanze:

- Quando la funzione di avvio termina.
- Quando un thread chiama la funzione **pthread_exit**
- Quando un thread chiamante termina.

7.3 Condivisione della memoria

I thread di un processo condividono le variabili globali, a patto che queste siano thread-safe ovvero se:

- Il loro valore è consistente quando viene letta da più thread contemporaneamente
- Il loro valore non viene alterato da un thread quando un altro la sta leggendo.

Poiché ogni thread ha la propria pila, **NON** condividono le variabili locali.

8 Concorrenza

È un **problema di sincronizzazione** di accesso alle risorse condivise.

Sono necessari meccanismi come mutex, condition variable, semafori, ecc...

8.1 Tipi di sincronizzazione

- **Per sezione critica:**
 - Quando la struttura viene modificata in un unico punto del codice.
 - Basta associare una mutex alla sezione critica.
- **Per struttura:**
 - Quando la struttura viene modificata in più punti del codice.
 - Necessario associare una mutex all'intera struttura.

8.2 Mutex Posix

- È una variabile di tipo `pthread_mutex_t`
- Può assumere due stati alternativi:
 - **locked** = il thread che ha acquisito il lock può proseguire.
 - **unlocked** = il thread che ha rilasciato il lock può proseguire.
- Può essere chiuso da un solo thread alla volta, gli altri sono bloccati. Quando il thread che ha acquisito il lock lo rilascia, allora uno dei thread bloccati può acquisirlo.

8.3 Deadlock

È la situazione in cui due o più thread sono bloccati in attesa di una risorsa che non verrà mai rilasciata.

Può essere evitato rispettando queste "regole":

- Ogni thread deve acquisire i lock sempre nello stesso ordine.
- Ogni thread deve rilasciare i lock sempre nello stesso ordine.

Non è sempre possibile.

8.4 Condition Variable

Meccanismo di sincronizzazione spesso usato con i mutex per garantire l'accesso sicuro e sincronizzato alle risorse condivise tra i thread.

- Un thread ottiene il mutex e testa il predicato.
- Se il predicato è verificato allora il thread esegue le sue operazioni e rilascia il mutex.
- Se il predicato non è verificato, in modo atomico:
 - Il mutex viene rilasciato (implicitamente).
 - Il thread si blocca sulla condition variable.
- Un thread bloccato riacquisisce il mutex nel momento in cui viene svegliato da un altro thread

9 Socket

Permette la comunicazione tra due processi (eseguiti sulla stessa macchina o su macchine diverse) attraverso la rete.

Viene identificata da un **IP** e un **numero di porta**. È un canale di comunicazione bidirezionale.

9.1 Domini e Stili di comunicazione

Il **dominio** di una socket equivale alla scelta di una famiglia di protocolli.

Ogni dominio è individuato da un intero non negativo.

Un dominio è `PF_INET` per la comunicazione tra processi che risiedono su macchine diverse e che utilizzano il protocollo TCP/IP con indirizzi IPv4.

Lo **stile di comunicazione** è individuato da costanti nella forma `SOCK_nomeStile`.

Un esempio è `SOCK_STREAM` per la comunicazione affidabile, orientata alla connessione, con flusso di byte.

9.2 Creazione

Il metodo `socket(domain, type, protocol)`:

- `domain` = dominio di comunicazione
- `type` = stile di comunicazione
- `protocol` = protocollo da utilizzare. Se è 0 allora viene scelto in base ai parametri precedenti.
- Restituisce il fd del socket creato o -1 in caso di errore.

9.3 Invio dati a un socket

Il metodo `write` è usata per scrivere dati su un file o un socket. Generalmente usato quando si lavora con file o si scrive su un socket in una connessione TCP.

Il metodo `send` è specifica per i socket. Simile a `write` ma include opzioni aggiuntive (come la possibilità di inviare flag). Generalmente usato quando si ha bisogno di un controllo più fine sulla comunicazione.

Il metodo `sendto` è simile a `send`, ma permette di specificare un indirizzo di destinazione. Utile per il protocollo UDP, dove l'indirizzo di destinazione potrebbe cambiare da un messaggio all'altro.

Il metodo `sendmsg` è una versione avanzata di `send` e `sendto` in quanto permette di inviare più messaggi contemporaneamente e di controllare vari aspetti del messaggio (come destinazione e le opzioni del protocollo).

9.4 Ricezione dati da un socket

Il metodo `read` usata per leggere dati da un file o un socket. Generalmente usato quando si lavora con file o si scrive su un socket in una connessione TCP.

Il metodo `recv` è specifico per i socket. Simile a `read` ma include opzioni aggiuntive (come la possibilità di inviare flag). Generalmente usato quando si ha bisogno di un controllo più fine sulla comunicazione.

Il metodo `recvfrom` è simile a `recv`, ma permette di specificare un indirizzo di origine. Utile per il protocollo UDP, dove l'indirizzo di origine potrebbe cambiare da un messaggio all'altro.

Il metodo `recvmsg` è una versione avanzata di `recv` e `recvfrom` in quanto permette di ricevere più messaggi contemporaneamente e di controllare vari aspetti del messaggio (come origine e le opzioni del protocollo).